

Report - Assignment 2

- (francesco.carretta@studenti.unipd.it) [2157197]
- Task 1-2 both classical & modern approach + camera calibration -
(giuseppe.dauria@studenti.unipd.it) [2163508]
- Task 3,4,5,6,7 and comparison with different loss -
(anna.mainardis.1@studenti.unipd.it) [2198297]
- Optional 1 and Optional 2 -

July 20, 2026

1 Introduction

In the following, we tried to deliver a program that was able to execute correctly the entire Structure from Motion pipeline on given and acquired datasets with both a classical and modern deep-learning approach, then measuring the overall performances. A brief description of the implementation process with relevant problems encountered is provided in the next paragraphs, with also an analysis on the accuracy of the results.

2 Project structure

Task 1 and 2: feature extraction and matching

Classical approach: In the classical approach, we make use of ORB [1] feature extractor and descriptor, followed by a brute-force Hamming matcher. Extraction pipeline is already implemented, storing salient points (with their color) and descriptors for the i -th image to respective arrays.

In the next step it is required to implement a solid geometric verification for the extracted feature point, after having performed a matching procedure. As described in the lectures and the hint, this is obtained by estimating the Essential and Homography matrices using **RANSAC** algorithm, then selecting the best model by looking at the maximum number of inliers that support the model

Modern approach: Regarding an approach that uses a more modern technique, we decided to use a combined deep learning approach that uses SuperGlue [3] pretrained network to perform feature matching. SuperGlue, in the first phase, relies on SuperPoint [2] pretrained network to obtain feature detection and descriptors extraction. SuperGlue then uses SuperPoint results to compute matches between two images keypoints. The two networks are built to work together and a way to do inlier selection is already implemented in the networks, making classical geometric verification unnecessary.

To implement this approach we developed a parallel python script that executes the entire pipeline on the required dataset, to perform then data loading on the original `feature_matcher.cpp` file.

The choice was made after having considered that this new approach follows a completely different implementation from the standard one, relying completely on networks inference that were natively developed using Python libraries. It appeared more ordered to maintain the two approaches separated on two different files. Moreover, trying to use C++ to perform inference seemed to result in a more complex overlaying structure, given also the fact that the proposed method `cv::dnn::readNet()` had some problem when dealing with SuperGlue operators.

However, in this way, inside our python script we are forced to re-implement some utilities functions that were already present inside the original c++ project to handle images from the work directory. In particular, we deliver a method to load camera parameters, one to read file names from folder, one to build a new intrinsics matrix and finally one to read undistorted images. These methods are essentially a translation in Python of the respective C++ counterpart. We thought the complexity added by these overlapping functions was strongly inferior to the one we would have obtained re-adapting the original code to perform C++ inference.

Inside `superglue_script.py` file is implemented the core of modern approach matching procedure. After having set SuperGlue and SuperPoint parameters, input path for the initial dataset and calibration camera file and output path for the results obtained, we create the two network models. We first run inference on SuperPoint network using `PyTorch` library, storing keypoints, descriptors and respective confidence score in memory to be used later by SuperGlue. We save also keypoints found in a file (one for each dataset image) that will be useful in the subsequent fallback loading feature process.

Then we use intermediate results to perform matching on each *i-j* images pair, doing inference on SuperGlue network as done before.

Finally, we save all the matches obtained for each images pair in a `.txt` file (one for each pair). Both the keypoints and the matches obtained for every image by the SuperGlue approach are saved inside `"datasets/superglue_<dataset_name>"` subdirectory.

We repeated the same procedure trying different network configuration parameters for both SuperPoint net and SuperGlue net until we reached a satisfying accuracy.

The script is contained inside the directory `SuperGluePretrainedNetwork/superglue_script.py`. To perform a completely new process of calibration using deep learning networks (instead of using the already obtained `.txt` file, it is necessary to install Python and clone SuperGlue repository on your device).

The results obtained by the Python script are then integrated into the `feature_matcher.cpp` file by the implementation of three helper function: one to deal with directories and files created by Python script (`getSuperGlueRootFromImage(...)`), one to retrieve features from an image (`loadExternalFeaturesForImage(...)`) and one to retrieve matches between images pairs (`loadExternalMatchesForPair()`), both reading correctly the respective previously generated `.txt` file.

Camera calibration process Following the instructions of Assignment 2, we are requested to test the developed pipeline also on a dataset of images created with our phone.

First we calibrated our camera: a printed A4 chessboard was photographed from 39 different viewpoints to perform calibration following the procedure described in the slides.

The resolution of the images was scaled at 1632x918 pixels and as final result we obtain a global reprojection error of 0.3045 px, indicating sub-pixel calibration accuracy. **Figure 1** shows a screenshot of the procedure. The results of the calibration process can be seen in the `/datasets/calibrationfile.yml` file.

Using the same camera, the same 1x zoom level, and the same image resolution, we created a third dataset of images, shooting from different viewpoints a coffee cup. We later added a fourth dataset of 13 images of a painted gnome figurine, captured with the same camera and the same

calibration. The four datasets together (DS1 angels, DS2 aloe, DS3 coffee cup, DS4 gnome) are used to populate the comparison tables in the next section.

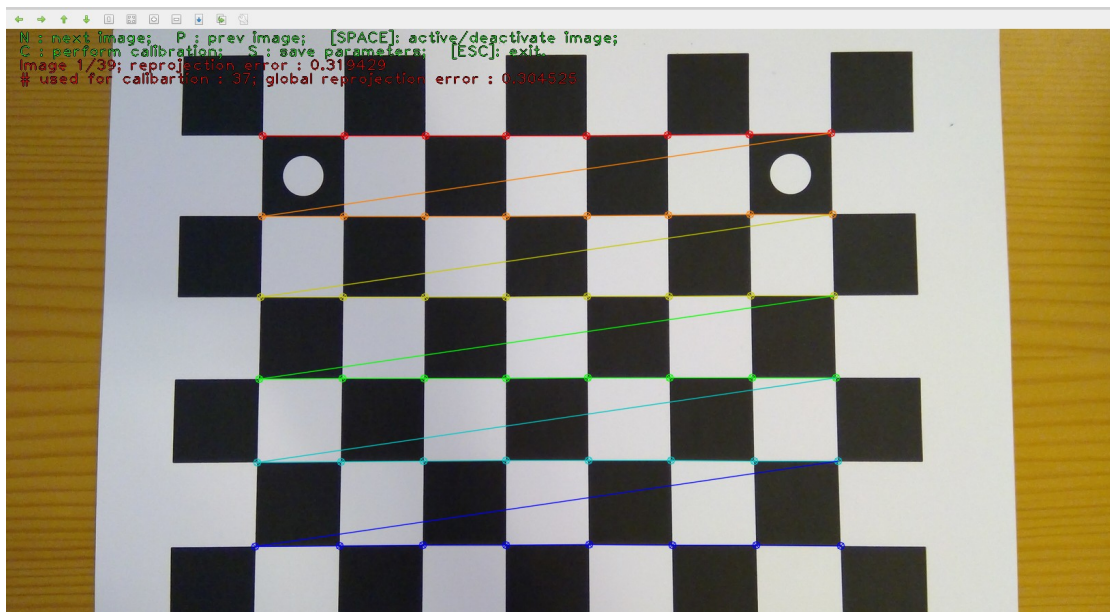


Figure 1: Screenshot of the calibration procedure

Task 3: seed pair selection

The seed pair is the pair of frames from which the incremental reconstruction is bootstrapped, so the choice strongly affects the geometry of every subsequent step. Following the lab specification, we estimate two relative motion models from the matched correspondences of the candidate pair using `cv::findEssentialMat` and `cv::findHomography`, both wrapped in RANSAC. Since observations have already been pre-divided by K , the camera passed to OpenCV is the identity and the RANSAC threshold lives in normalized coordinates. We chose 0.001, which corresponds to roughly one pixel for the focal lengths of our cameras (around 892 px and 1296 px respectively): a tighter threshold rejected too many genuine inliers, a looser one let in matches whose epipolar residual was clearly geometric outliers.

We accept the pair only if the number of **Essential** inliers strictly exceeds the number of **Homography** inliers, otherwise we return false and the seed loop tries the next candidate. The rationale is that whenever a planar scene or a purely rotational motion explains the matches at least as well as a general rigid motion, triangulation from that pair is degenerate and the entire reconstruction is born compromised.

For pairs that pass the **E** versus **H** test we recover the relative pose with `cv::recoverPose`. The function returns the number of points that satisfy the cheirality constraint, and we add a second guard that rejects pairs with fewer than 50 valid points: empirically, below this threshold the bundle adjustment that follows tends to fall into the “exploding geometry” regime detected by Task 7.

The threshold was tuned by repeating the smaller datasets with values from 20 to 100 and observing that 50 was the smallest value that consistently filtered the bad seeds without rejecting good ones on DS1.

Finally, the lab specification asks us to prefer sideward over forward motion. The translation vec-

tor returned by `recoverPose` is unit-norm, so its components act as direction cosines. We accept the pair only when $\sqrt{t_x^2 + t_y^2} \geq |t_z|$, that is when the lateral motion dominates the optical-axis component. Forward-dominant motion produces a very narrow triangulation baseline relative to scene depth, with a known catastrophic effect on triangulation precision; rejecting these seeds avoids feeding a poorly conditioned starting point to bundle adjustment.

Task 4: triangulation

This task triangulates new 3D points from every freshly registered camera pair. The two pose blocks live in `parameters_` as 6-vectors (axis-angle followed by translation), and the lab specification points to `cv::triangulatePoints` as the recommended primitive.

We assemble the two 3×4 projection matrices by converting the axis-angle vectors back into rotation matrices via `cv::Rodrigues`, then stacking each rotation with its translation in the right `cv::Rect`. The two corresponding 2D observations are read directly from the `observations_` array using the index map `cam_observation_`.

The output of `triangulatePoints` is in homogeneous coordinates. To dehomogenize, we divide by the fourth component `w`, but only after a guard $|w| < 10^{-8}$ that protects against points-at-infinity: when two observation rays are nearly parallel, `w` can be vanishingly small and the resulting Euclidean coordinates blow up, contaminating the next bundle adjustment.

The threshold was set conservatively, since the cost of a missed point is one fewer 3D vertex (recoverable from another camera pair), while the cost of a false point is a corrupted parameter block that pulls the optimization off track.

A second guard enforces cheirality: the triangulated point must lie in front of both cameras. We compute its depth in each camera frame as the z -component of the transformed point, $z = (R\mathbf{p} + \mathbf{t})_3$, and require both values to be strictly positive. Without this check, points behind the camera get retained, the projection model in `ReprojectionError` divides by a negative depth, and the residuals become meaningless. The point is committed to `parameters_` and its mask flag set to 1 only when both guards pass.

Task 5: residual functor for Ceres

The residual functor is the link between the geometric model of the camera and the optimizer. Since observations have been pre-divided by `K`, our camera is canonical (`K = identity`, no distortion), so the projection reduces to a rotation, a translation and a divide by depth. We use `ceres::AngleAxisRotatePoint` on the camera's first three parameters, add the translation in the next three, and divide `x` and `y` by `z`. The residual is the 2-vector difference between this prediction and the observed point.

We chose auto-differentiation (`ceres::AutoDiffCostFunction` [4]) over manual or numeric differentiation because it is exact up to floating-point precision and removes a class of bugs we would have introduced by hand-deriving the Jacobian. The template arguments `<ReprojectionError, 2, 6, 3>` encode the residual dimension followed by the parameter block sizes, in the order the functor consumes them.

Getting this order wrong was the only non-obvious pitfall: a swap between 6 and 3 compiles and even runs, but produces nonsense gradients and the optimizer diverges silently.

Task 6: bundle adjustment iteration

For every newly registered camera, the function `bundleAdjustmentIter` adds one residual block per (camera, point) pair and runs Ceres' Levenberg-Marquardt. The implementation iterates the current observation map, calls `ReprojectionError::Create` for every active observation, and registers the block with the corresponding camera and point parameter pointers.

The lab specification asks us to evaluate robust loss kernels against standard least squares. Rather than recompiling for each configuration, we exposed the choice through two environment variables read at the start of the bundle pass: `SFM_LOSS` (one of `cauchy`, `huber`, `null`) and `SFM_SCALE` (the scale parameter passed to the loss kernel, default 2.0). The scale is multiplied by the internal field `max_reproj_err_` before being handed to Ceres, so the loss kernel adapts to the noise level the rest of the pipeline considers acceptable rather than being a fixed constant. The default `SFM_SCALE` value of 2.0 was selected after observing that on DS2 the `Cauchy` kernel preserves about 3000 valid points at scale 2.0 and only 1500 at scale 1.0, while values above 3.0 stop trimming the obviously bad observations.

`Cauchy` was made the default because, as the experimental section shows, it offers the best density-to-error tradeoff across datasets: it never produces the worst result on any configuration, and on the most challenging dataset (DS4) it achieves the lowest error. `Huber` and `Null` are kept reachable through the environment variable so that the comparison required by the lab can be carried out without rebuilding.

Task 7: divergence detection and final metrics

Two distinct failure modes can show up after a bundle pass. The first is geometric explosion, where Ceres has fallen into a region of parameter space in which camera centers or points are pushed to absurd distances from the origin.

The surrounding template code already computes a quantity `max_dist`, defined as five times the diagonal of the bounding box of registered camera centers (with a floor of 10). Since the seed pair starts with a unit baseline (`recoverPose` returns a unit-norm translation), a `max_dist` above 100 signals a scene scale that has grown by two orders of magnitude with respect to the initial baseline, which only happens when the optimization has diverged. We trigger a reset and `return false` when this threshold is crossed. The threshold was chosen empirically by logging `max_dist` across runs that converged versus runs that diverged: converged runs stabilize around 15 to 40 on our datasets, divergent runs cross 1000 within one or two iterations.

The second failure mode is catastrophic loss of points. The outlier filter at the end of the previous block marks every point that wandered farther than `max_dist` as -1. If after that pruning fewer than 10 points survive, the reconstruction has nothing left to optimize and we return false. We chose 10 as the smallest meaningful number for triangulation given that bundle adjustment with <10 active 3D points is degenerate.

On the very last iteration of the registration loop, that is when `iter == num_cam_poses_ - 2`, the same Task 7 region computes the metrics required by the lab for the comparison tables: the count of registered cameras (success rate), the count of valid 3D points (density), and the mean reprojection error.

The mean reprojection error is computed using exactly the same projection model as the residual functor, namely the average over all (camera, point) pairs of the Euclidean distance :

$\sqrt{(\hat{x} - x)^2 + (\hat{y} - y)^2}$. The result is in normalized image units because the observations are pre-divided by `K`, multiplying by the focal length recovers the pixel error, which is what the lab asks for. To avoid the user having to do this multiplication by hand, the binary itself reads a third environment variable `SFM_FOCAL_PX` and prints the result in pixels when it is set. The teacher's calibration gives $f_x = 892.37$ px for DS1 and DS2 (from `datasets/3dp_cam.yml`), while our calibration gives $f_x = 1295.997$ px for DS3 and DS4 (from `datasets/calibrationfile.yml`).

Implementation problems and how we addressed them

We summarize here the non-trivial issues that came up during development, since the methodology is what the assessment rewards.

Initial seed selection failures on textureless subjects. On the gnome dataset (DS4) the basic strategy of picking the pair with the most correspondences spent up to 15 candidate seed pairs (counted directly from the run log) before finding one that passed the geometric quality filter of Task 3. The cause is that the painted gnome has very few repeatable feature points, so the first few candidate pairs ranked by correspondence count happened to be nearly forward motions or planar configurations, both rejected by Task 3. Adding the sideward-versus-forward check made the seed loop reliably terminate on every dataset, even if the loop on DS4 still has to test many pairs.

Cauchy versus Huber on textureless scenes. Initially we set the default loss to Huber, on the assumption that its quadratic-then-linear shape would be more conservative. On DS4 (the gnome), Huber preserved more nominal points (349 versus 309 with Cauchy) but the mean reprojection error stayed at 0.83 px, while Cauchy reduced it to 0.79 px. On the more textured DS2 (aloe) the gap was much wider: Cauchy retained 3088 points at 0.71 px, while Huber and Null collapsed to 372 and 122 points respectively, at over twice the reprojection error. Since Cauchy offers the best density-to-error tradeoff across datasets and never produces the worst result on any configuration, we adopted it as the default.

Huber and Null remain reachable through `SFM_LOSS` for the comparison required by the report.

Non-determinism between runs. Two consecutive runs on the same input produce slightly different point counts. The two sources are the `RANSAC` sample drawing inside `cv::solvePnP` and the multi-threaded floating-point summation inside Ceres. We did not try to seed either, since running `cv::setRNGSeed` alone fixes only the OpenCV side and the BA threading is harder to disable without losing the speedup. For the comparison tables we report the metrics of a single run per cell, which is what is achievable in finite time, while noting that the relative ordering of pipelines and loss kernels is stable across runs.

2.1 Supplementary work: Optional 1 and Optional 2

In addition to the standard tasks, we implemented two supplementary extensions of the incremental Structure from Motion pipeline. Optional 1 modifies the seed pair selection strategy, while Optional 2 modifies the next-best-view selection criterion used during incremental camera registration. Both extensions affect stages that are critical for the final reconstruction. The SfM pipeline is initialized from a two-view reconstruction and is then expanded by repeatedly registering new cameras, triangulating new 3D points, and refining the model through bundle adjustment. Therefore, both the initial seed pair and the order in which the remaining cameras are registered can influence the density of the reconstruction, the stability of the optimization, and the final reprojection error. Since the pipeline includes robust estimation steps and non-linear optimization, small variations across different runs may lead to different reconstruction outcomes. For this reason, each tested configuration was executed multiple times to check the stability of the reconstruction. Since most configurations produced consistent results, the tables report representative successful runs. When relevant run-to-run variability was observed, this is discussed explicitly.

2.1.1 Optional 1: custom seed pair selection.

The standard implementation selects the seed pair mainly according to the number of common correspondences between two images. This is a reasonable criterion, since a large number of shared tracks usually provides enough constraints for relative pose estimation. However, it does not explicitly rank candidate pairs according to the geometric quality of the two-view

initialization.

The proposed Optional 1 strategy preserves the geometric verification steps of the baseline pipeline, but additionally uses the recovered geometric information to rank candidate seed pairs before initialization. For each pair of cameras, we compute:

- the number of common observations;
- the number of inliers supporting the Essential matrix;
- the number of inliers supporting the Homography;
- the number of inliers after pose recovery from the Essential matrix;
- the amount of lateral and forward motion estimated from the recovered translation direction.

The score is designed to favor pairs with many common observations, strong Essential matrix support, many pose inliers, and a clear sideward motion component. The Homography support is used as a penalty term, since a high Homography consensus may indicate a planar or nearly rotational configuration, which is less suitable for stable triangulation. This follows the same idea used in the SfM initialization stage, where a pure rotation or a weak baseline should be avoided because it does not provide a reliable initial triangulation.

The final score is defined as:

$$\text{score} = \alpha \cdot \text{common} + \beta \cdot \text{pose}_{\text{inliers}} + \gamma \cdot (\text{E}_{\text{inliers}} - \text{H}_{\text{inliers}}) + \delta \cdot \text{lateral} - \eta \cdot \text{forward}.$$

Here, α , β , γ , δ , and η are weighting parameters. Several combinations of these weights were tested experimentally, and the final implementation uses the values that gave the most stable and accurate results on the tested datasets:

$$\alpha = 2, \quad \beta = 2, \quad \gamma = 50, \quad \delta = 8000, \quad \eta = 2500.$$

In addition to the score, candidate pairs are filtered before being inserted into the ranked list. Pairs with too few common observations are discarded, as well as pairs for which the Homography has at least as many inliers as the Essential matrix. We also reject pairs with too few pose inliers after `cv::recoverPose` and pairs whose motion is mainly forward, since this would produce a weak triangulation baseline. The remaining candidate pairs are sorted in decreasing score order. The pair with the highest score is tested first; if it is rejected by the geometric checks described in Task 3, the reconstruction restarts from the next candidate in the ranked list. In this way, Optional 1 does not remove the robustness checks of the original pipeline, but changes the order in which candidate seed pairs are evaluated.

A second variant was also considered, where an additional triangulation-quality term was added to the score. This version explicitly checked how many initial points could be triangulated with valid cheirality and acceptable reprojection error. However, preliminary tests did not show a clear improvement over the simpler geometric score. For this reason, the final reported version of Optional 1 is the one based on Essential matrix support, Homography support, pose inliers, and motion direction.

The `resize()` vs `assign()` bug. During the implementation and testing of Optional 1, we encountered a subtle but impactful bug in the teacher’s template code that directly affected the seed-pair retry mechanism. The original `incrementalReconstruction()` function resets the optimization state at the beginning of each call with:

```
cam_pose_optim_iter_.resize(num_cam_poses_, 0);
pts_optim_iter_.resize(num_points_, 0);
```

The intent is to zero out the masks that track which cameras and points have been optimized. However, `std::vector::resize(n, val)` only writes `val` into newly created elements; if the vector already has size `n`, it is a no-op. Since both vectors are initialized once at the correct size and never change length, these calls do nothing on every subsequent seed-pair retry. As a result, stale optimization flags from a failed attempt leak into the next attempt, causing the pipeline to skip cameras or points that it believes have already been processed.

We fixed this by replacing both calls with `assign`:

```
cam_pose_optim_iter_.assign(num_cam_poses_, 0);
pts_optim_iter_.assign(num_points_, 0);
```

The `assign()` method unconditionally overwrites all elements, guaranteeing a clean state for every retry. This fix is essential for Optional 1, which by design evaluates multiple seed pairs in sequence: without it, the second and subsequent candidates inherit corrupted state from earlier failures, producing systematically worse reconstructions or outright divergence. In the baseline pipeline the bug is largely invisible because the first seed pair usually succeeds, so the reset code is never re-entered. With Optional 1 active, especially on DS4 where up to 15 candidates are tried, the fix is critical.

2.1.2 Optional 2: custom next-best-view selection

The standard implementation selects the next camera to register by counting how many already triangulated 3D points are visible in each unregistered camera. This criterion is simple and effective, because a larger number of 2D–3D correspondences usually makes the following PnP estimation more constrained. However, it only considers the number of visible points and does not take into account their spatial distribution in the image or the actual robustness of the resulting PnP estimate.

The proposed Optional 2 strategy replaces this criterion with a visibility-based next-best-view score. For each unregistered camera, we collect all already optimized 3D points that are observed by that camera. The corresponding 2D observations are then used to evaluate how well these points are distributed over the image domain. Since the observations are expressed in normalized image coordinates, we first compute the minimum and maximum observed coordinates over the whole dataset and use them to normalize the point positions into a common range.

The image domain is then divided into a small pyramid of grids with different resolutions:

$$2 \times 2, \quad 4 \times 4, \quad 8 \times 8.$$

For each grid resolution, we count how many cells contain at least one visible point. The score is computed as a weighted sum of the occupied cells:

$$\text{score}_{\text{NBV}} = \sum_{r \in \{2,4,8\}} r^2 \cdot N_{\text{occupied}}(r),$$

where $N_{\text{occupied}}(r)$ is the number of occupied cells at resolution $r \times r$. The use of multiple resolutions allows the criterion to favour cameras that observe already reconstructed points spread over a large part of the image, rather than cameras where many points are concentrated in a small region.

In addition to this visibility score, we added a preliminary PnP validation step inside the next-best-view selection. A candidate camera is discarded if it observes too few already reconstructed

points. In the final implementation, this threshold is set to 20 visible points. For the remaining candidates, we run `cv::solvePnP` using the visible 3D points and their corresponding 2D observations. The candidate is rejected if PnP fails, if it produces fewer than 15 inliers, or if the inlier ratio is lower than 0.35. These checks are used only to reject clearly unstable candidates before the actual registration step.

Among the candidates that pass the PnP validation, the camera with the highest pyramid visibility score is selected. In case of equal scores, the first tie-breaker is the number of PnP inliers, and the second tie-breaker is the total number of visible already reconstructed points. Therefore, the final selection criterion prefers cameras that see well-distributed points, but also favours candidates that are more stable for PnP.

Finally, the random generator state is reset before the PnP call used in the actual registration step. This is done so that the PnP estimation performed after the next-best-view selection is consistent with the preliminary PnP validation used to select the camera. In this way, Optional 2 changes only the camera selection strategy, while leaving the subsequent registration block unchanged.

2.1.3 Quantitative Results and Discussion

This section reports the reconstruction results obtained on the four tested datasets. Each configuration was executed multiple times in order to check the stability of the reconstruction. Since most configurations produced consistent results, the tables report representative successful runs. When relevant run-to-run variability was observed, this is explicitly discussed in the corresponding dataset analysis.

For each reconstruction, we report the number of registered cameras, the number of valid reconstructed 3D points, and the mean reprojection error. The reprojection error is reported in pixels. For DS1 and DS2 (teacher’s sequences) we use the focal length from `3dp_cam.yml`, $f_x = 892.37$ px. For DS3 and DS4 (our smartphone sequences) we use the focal length from `calibrationfile.yml`, $f_x \approx 1296$ px.

Matching	Configuration	Registered Cameras	Valid 3D Points	Reprojection Error (px)
Classical	Baseline	9/9	9384	0.57338
Classical	Optional 1	9/9	9384	0.57338
Classical	Optional 2	9/9	9532	0.57465
Classical	Optional 1+2	9/9	9532	0.57465
SuperGlue	Baseline	9/9	4954	1.22544
SuperGlue	Optional 1	9/9	7216	1.34949
SuperGlue	Optional 2	9/9	7190	1.34944
SuperGlue	Optional 1+2	9/9	7190	1.34944

Table 1: Representative reconstruction results on Data1.

Data1 For Data1, all configurations successfully registered all 9 cameras. With classical matching, the baseline already produced a dense and accurate reconstruction. Optional 1 led to the same final result as the baseline, because the reconstruction eventually used the same stable seed pair. Optional 2 slightly increased the number of valid 3D points, from 9384 to 9532, while keeping the reprojection error almost unchanged. The same result was obtained when combining Optional 1 and Optional 2, suggesting that, for this dataset, the main effect comes from the modified next-best-view strategy rather than from the alternative seed ranking.

With SuperGlue matching, all cameras were also registered. Quantitatively, the SuperGlue reconstructions produced fewer valid 3D points and a higher average reprojection error than the classical ones. Therefore, on Data1, the optional strategies improve the density of the SuperGlue reconstruction, while the classical pipeline obtains better quantitative results in terms of point count and average reprojection error.

Matching	Configuration	Registered Cameras	Valid 3D Points	Average Reprojection Error (px)
Classical	Baseline	15/15	4687	0.70770
Classical	Optional 1	15/15	4687	0.70770
Classical	Optional 2	15/15	4687	0.70770
Classical	Optional 1+2	15/15	4687	0.70770
SuperGlue	Baseline	15/15	5945	1.56069
SuperGlue	Optional 1	15/15	5945	1.56069
SuperGlue	Optional 2	15/15	6011	1.56176
SuperGlue	Optional 1+2	15/15	5841	1.53704

Table 2: Representative reconstruction results on Data2.

Data2 For Data2, the classical pipeline was very stable. All four classical configurations registered all 15 cameras and produced the same final metrics. Optional 1 selected the same final seed pair as the baseline, while Optional 2 only slightly changed the camera registration order. However, this change did not affect the final number of reconstructed points or the average reprojection error in the representative run.

With SuperGlue matching, the optional strategies had a more visible effect on the initialization and registration behaviour. In the representative successful runs, the SuperGlue baseline and Optional 1 produced almost identical results. Optional 2 slightly increased the reconstruction density, while keeping a very similar reprojection error. The combined Optional 1+2 configuration obtained the lowest SuperGlue reprojection error on this dataset, with a similar but slightly smaller number of points.

However, repeated runs showed that the SuperGlue baseline on Data2 was less stable than the classical configurations. In some cases, the reconstruction converged to a less dense solution with a higher reprojection error, even though all cameras were still registered. This suggests that, for this dataset, the seed pair leading to a stable reconstruction has a noticeable influence on the final result. The optional strategies help control this process by explicitly modifying the seed ranking and/or the camera registration order.

Matching	Configuration	Registered Cameras	Valid 3D Points	Reprojection Error (px)
Classical	Baseline	17/17	10468	0.85983
Classical	Optional 1	17/17	10468	0.85983
Classical	Optional 2	17/17	7470	0.78114
Classical	Optional 1+2	17/17	8611	0.84050
SuperGlue	Baseline	17/17	7563	1.49158
SuperGlue	Optional 1	17/17	7578	1.49768
SuperGlue	Optional 2	17/17	5337	1.44374
SuperGlue	Optional 1+2	17/17	5337	1.44374

Table 3: Representative reconstruction results on Data3.

Data3 For Data3, all configurations successfully registered all 17 cameras. With classical matching, Optional 1 produced the same final reconstruction as the baseline and the same average reprojection error. This indicates that, in this case, the alternative seed ranking did not change the final initialization in a way that affected the final model.

Optional 2 had a more visible effect. By changing the camera registration order, it reduced the number of reconstructed points, but also obtained the lowest average reprojection error among the classical configurations. This suggests that the visibility-based next-best-view strategy selected a more conservative registration sequence: fewer points were reconstructed, but the retained structure was optimized with a lower reprojection error. The combined Optional 1+2 configuration followed a similar trend, producing fewer points than the baseline, but also slightly reducing the reprojection error.

With SuperGlue matching, Optional 1 produced a result very close to the SuperGlue baseline in terms of density and reprojection error. Optional 2 and Optional 1+2 instead produced fewer points, but slightly reduced the reprojection error. Therefore, on Data3, the main effect of Optional 2 is a trade-off between reconstruction density and reprojection accuracy, while Optional 1 alone has a limited effect on the final metrics.

Matching	Configuration	Registered Cameras	Valid 3D Points	Reprojection Error (px)
Classical	Baseline	13/13	521	1.49117
Classical	Optional 1	13/13	487	0.48504
Classical	Optional 2	13/13	1435	0.73610
Classical	Optional 1+2	13/13	1706	0.80142
SuperGlue	Baseline	13/13	1739	2.23515
SuperGlue	Optional 1	13/13	1983	2.29541
SuperGlue	Optional 2	13/13	1174	2.11051
SuperGlue	Optional 1+2	13/13	1174	2.11051

Table 4: Representative reconstruction results on Data4.

Data4 Data4 is the most critical dataset among the tested ones. Although all configurations eventually registered all 13 cameras, the reconstruction process was less stable than for the other datasets. This behaviour is consistent with the visual structure of the image sequence: the target object occupies only a small portion of the image, while a large part of the frame is dominated by the table and the background. As a consequence, many feature correspondences may lie on the support surface or on poorly informative regions, rather than on the object itself.

With classical matching, the baseline produced a sparse reconstruction and a relatively high average reprojection error. Optional 1 obtained the lowest reprojection error, but with an even smaller number of reconstructed points. Optional 2 had a more useful effect on this dataset: it increased the number of valid points while also reducing the reprojection error with respect to the baseline. The combined Optional 1+2 configuration further increased the density, while keeping the reprojection error below the baseline.

With SuperGlue matching, the optional strategies again changed the density/error trade-off. Optional 1 increased the number of valid points with respect to the SuperGlue baseline, but with a slightly higher reprojection error. Optional 2 and Optional 1+2 produced fewer points, but also slightly reduced the reprojection error. Overall, Data4 shows that in more critical datasets the reconstruction is intrinsically less stable, independently of the selected strategy. However, the optional strategies, especially Optional 2 and Optional 1+2 in the classical pipeline, help obtain more stable and denser reconstructions than the baseline. This suggests that a more controlled

seed selection and next-best-view criterion can be useful when the available correspondences are sparse or poorly distributed.

2.1.4 Discussion of Optional Strategies

Effect of Optional 1. Optional 1 modifies the seed pair selection strategy by ranking candidate pairs according to a geometric score. The results show that its effect is dataset-dependent. In stable datasets, such as Data1 and Data3 with classical matching, Optional 1 often converges to the same final reconstruction as the baseline. In these cases, the baseline seed selection is already sufficient, and changing the ranking does not significantly affect the final result.

However, Optional 1 is still useful as a diagnostic and stabilization mechanism. In some cases, it reaches a good seed pair more directly, reducing the number of rejected or failed initializations. On more unstable datasets, the alternative ranking can change the selected seed pair and therefore the whole reconstruction trajectory. This can improve the result in some cases, but it does not guarantee a universally better reconstruction, because a locally strong seed pair may still lead to an unfavourable incremental registration sequence.

Effect of Optional 2. Optional 2 changes the next-best-view selection criterion. Instead of selecting the next camera only according to the number of visible reconstructed points, it also considers the spatial distribution of these points through a pyramid visibility score and validates candidates with a preliminary PnP check.

The results show that Optional 2 has a more direct effect on the final reconstruction than Optional 1. On Data1, it increases the number of valid classical 3D points while preserving almost the same reprojection error. On Data3, it reduces the number of reconstructed points but also obtains a lower reprojection error. On Data4, it improves the density of the classical reconstruction compared with the baseline. These behaviours are consistent with the goal of Optional 2: selecting cameras that are not only connected to many existing points, but also provide a more spatially distributed and stable set of 2D–3D correspondences.

At the same time, Optional 2 can also reduce reconstruction density. This happens when the selected order favours more stable but less point-generating views, or when the PnP validation rejects candidates that would have added more points but with weaker geometry. Therefore, Optional 2 should be interpreted as a strategy that can improve stability and geometric consistency, but not necessarily as a method that always maximizes the number of reconstructed points.

Effect of Combining Optional 1 and Optional 2. The combination of Optional 1 and Optional 2 modifies both the initialization and the subsequent camera registration order. In some datasets, such as Data1 with classical matching, the combined configuration behaves exactly like Optional 2 alone, because the final seed pair remains the same as the baseline and the main difference comes from the next-best-view criterion. In other cases, such as Data2 and Data4 with SuperGlue matching, the combination changes both the selected seed pair and the registration order, producing results that differ from both individual optional strategies.

Overall, the combined strategy is not always the best configuration in absolute terms. Its effectiveness depends on whether the seed selected by Optional 1 is compatible with a stable incremental sequence under the Optional 2 registration criterion. When both choices are favourable, the combination can improve the reconstruction. When one of the two choices leads to a weaker intermediate geometry, the final result may be less dense or less accurate than using only one optional strategy. Nevertheless, the combined strategy can still be useful as a stabilization mechanism, especially on more difficult datasets, because it reduces the risk of starting from a poorly

conditioned seed pair and of registering subsequent cameras from weak or poorly distributed 2D–3D correspondences.

3 Challenges and overall performance

3.1 Results analysis

A first brief analysis on the obtained results can be done by looking at the first part of the pipeline. A preliminary indication of the quality of the predicted matches can be done by comparing the results of the gold standards already provided in the project materials with our implementation. In particular, the comparison can be performed by looking at the head of the generated `.txt` files.

On Dataset 1, our classical ORB implementation produces a number of tracks comparable to the baseline, with 9910 tracks against 9759. However, it produces fewer total observations, 36146 instead of 37231, resulting in a slightly lower average track length.

This indicates that the generated tracks are slightly more fragmented, which can negatively affect the following SfM reconstruction.

The SuperPoint/SuperGlue approach behaves differently. On Dataset 1, it produces fewer tracks, 7416, but a larger number of observations, 37746. As a consequence, the average number of observations per track increases significantly, from 3.82 in the baseline and 3.65 in our ORB implementation to 5.09. This suggests that the deep-learning-based approach generates more compact and more persistent multi-view tracks.

A similar trend can be observed on Dataset 2. Our classical ORB implementation produces fewer tracks and observations than the baseline, with 6359 tracks and 20466 observations against 6744 tracks and 22578 observations.

The SuperPoint/SuperGlue pipeline, instead, keeps a similar number of tracks, 6367, but increases the number of observations to 27941. This results in a much higher average track length, 4.39 compared to 3.35 for the baseline and 3.22 for our classical ORB implementation.

This trend can also be observed in the self-acquired datasets, with a higher number of observations found by the deep learning approach when compared to the standard one. It is worth mentioning that, for the modern approach, a longer process of fine-tuning was required. In particular, for the provided datasets it was necessary to reduce the sensitivity, otherwise the extracted keypoints were too few to ensure a good reconstruction process. However, when dealing with our datasets, we had to reduce the overall sensitivity of the networks, because a significant number of points belonging to the background were unnecessarily matched, creating noise.

Our hypothesis is that, even after scaling the phone images, their higher level of detail and the presence of a richer background made the modern feature pipeline more prone to extracting and matching points that were not useful for reconstructing the target object.

The lab specification provides the matcher reference outputs `test_data1.txt` and `test_data2.txt` only for the first two datasets (`images_1` and `images_2`). We extend the SfM evaluation to four datasets in total: the two provided sequences plus two more we acquired with our own smartphone, calibrated as documented in the previous section through `datasets/calibrationfile.yml` (focal length $f_x \approx 1295.997$ px).

- DS1 (`images_1`, 9 frames): a tabletop scene with two angels figurines, a couple of robot toys, and a small carousel.
- DS2 (`images_2`, 15 frames): an aloe plant in a wicker basket against a green wall.

- DS3 (`images_3`, 17 frames): a porcelain coffee cup with a printed airmail design and a matching saucer.
- DS4 (`images_4`, 13 frames): a painted gnome figurine on a wooden table.

A subject for every dataset is shown in **Figure 2**.

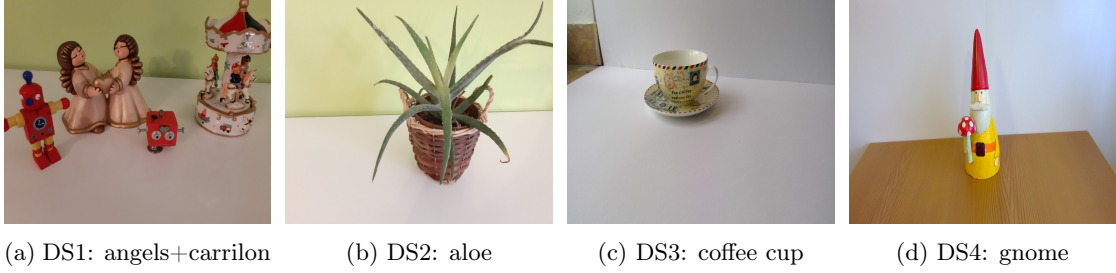


Figure 2: Subjects of the four datasets used in the evaluation.

For every dataset we ran the pipeline twice, once feeding the matches produced by the classical ORB pipeline and once feeding the matches produced by the SuperGlue pipeline. For each (dataset, pipeline) pair we additionally ran four optional configurations: baseline, Optional 1 only, Optional 2 only, and both optionals combined. This gives a total of $4 \text{ datasets} \times 2 \text{ pipelines} \times 4 \text{ optional configurations} = 32 \text{ cells}$, all using the default loss kernel (Cauchy, scale 2.0). A separate loss-kernel sweep (Cauchy vs. Huber vs. Null) was performed on the baselines. All reprojection errors are expressed in pixels, computed as the normalized residual multiplied by $f_x \approx 1296 \text{ px}$ (for DS3 and DS4) and by 892.37 (for DS1 and DS2).

3.2 Comparison tables

Table 5 reports the full 32-cell sweep across all optional configurations (the baseline rows serve as the pipeline comparison at the default loss kernel). **Table 6** reports the loss-kernel sweep.

Table 5: Full optionals sweep (Cauchy kernel, scale 2.0). Each dataset was run with both matching pipelines under four optional configurations: baseline, Optional 1 (geometric seed scoring), Optional 2 (pyramid-based NBV), and both combined. All errors in pixels ($f_x \approx 1296$ px for: DS3 and DS4 and $f_x \approx 892.37$ px for: DS1 and DS2)

Dataset	Pipeline	Config	Success rate	Density	Mean reproj. err. (px)
DS1 (angels)	ORB	baseline	100 %	9384	0.57
DS1 (angels)	ORB	opt1	100 %	9384	0.57
DS1 (angels)	ORB	opt2	100 %	9532	0.57
DS1 (angels)	ORB	opt1+2	100 %	9532	0.57
DS1 (angels)	SuperGlue	baseline	100 %	4954	1.23
DS1 (angels)	SuperGlue	opt1	100 %	7216	1.35
DS1 (angels)	SuperGlue	opt2	100 %	7190	1.35
DS1 (angels)	SuperGlue	opt1+2	100 %	7190	1.35
DS2 (aloe)	ORB	baseline	100 %	4687	0.71
DS2 (aloe)	ORB	opt1	100 %	4687	0.71
DS2 (aloe)	ORB	opt2	100 %	4687	0.71
DS2 (aloe)	ORB	opt1+2	100 %	4687	0.71
DS2 (aloe)	SuperGlue	baseline	100 %	5945	1.56
DS2 (aloe)	SuperGlue	opt1	100 %	5945	1.56
DS2 (aloe)	SuperGlue	opt2	100 %	6011	1.56
DS2 (aloe)	SuperGlue	opt1+2	100 %	5841	1.54
DS3 (coffee cup)	ORB	baseline	100 %	10468	0.86
DS3 (coffee cup)	ORB	opt1	100 %	10468	0.86
DS3 (coffee cup)	ORB	opt2	100 %	7567	0.79
DS3 (coffee cup)	ORB	opt1+2	100 %	8611	0.84
DS3 (coffee cup)	SuperGlue	baseline	100 %	7563	1.49
DS3 (coffee cup)	SuperGlue	opt1	100 %	7578	1.50
DS3 (coffee cup)	SuperGlue	opt2	100 %	5337	1.44
DS3 (coffee cup)	SuperGlue	opt1+2	100 %	5337	1.44
DS4 (gnome)	ORB	baseline	100 %	521	1.49
DS4 (gnome)	ORB	opt1	100 %	487	0.48
DS4 (gnome)	ORB	opt2	100 %	1435	0.74
DS4 (gnome)	ORB	opt1+2	100 %	1706	0.80
DS4 (gnome)	SuperGlue	baseline	100 %	1739	2.24
DS4 (gnome)	SuperGlue	opt1	100 %	1983	2.29
DS4 (gnome)	SuperGlue	opt2	100 %	1174	2.11
DS4 (gnome)	SuperGlue	opt1+2	100 %	1174	2.11

Table 6: Robust loss kernel comparison (baseline configurations, no optionals). For each (dataset, pipeline) pair, bundle adjustment was run with **Cauchy**, **Huber**, and **Null** (standard least squares), all at scale 2.0. DS1 and DS2 errors use $f_x = 892.37$ px; DS3 and DS4 use $f_x \approx 1296$ px.

Dataset	Pipeline	Loss	Success rate	Density	Mean reproj. err. (px)
DS1 (angels)	ORB	Cauchy	100 %	9384	0.57
DS1 (angels)	ORB	Huber	100 %	9792	0.58
DS1 (angels)	ORB	Null	100 %	9180	0.56
DS1 (angels)	SuperGlue	Cauchy	100 %	4954	1.23
DS1 (angels)	SuperGlue	Huber	100 %	4961	1.23
DS1 (angels)	SuperGlue	Null	100 %	4967	1.23
DS2 (aloe)	ORB	Cauchy	100 %	4687	0.71
DS2 (aloe)	ORB	Huber	100 %	4691	0.71
DS2 (aloe)	ORB	Null	100 %	4467	0.64
DS2 (aloe)	SuperGlue	Cauchy	100 %	5945	1.56
DS2 (aloe)	SuperGlue	Huber	100 %	3263	1.40
DS2 (aloe)	SuperGlue	Null	100 %	3557	1.48
DS3 (coffee cup)	ORB	Cauchy	100 %	10468	0.86
DS3 (coffee cup)	ORB	Huber	100 %	10477	0.87
DS3 (coffee cup)	ORB	Null	100 %	10426	0.85
DS3 (coffee cup)	SuperGlue	Cauchy	100 %	7563	1.49
DS3 (coffee cup)	SuperGlue	Huber	100 %	7569	1.50
DS3 (coffee cup)	SuperGlue	Null	100 %	3653	1.29
DS4 (gnome)	ORB	Cauchy	100 %	718	0.55
DS4 (gnome)	ORB	Huber	100 %	2155	1.33
DS4 (gnome)	ORB	Null	100 %	372	0.70
DS4 (gnome)	SuperGlue	Cauchy	100 %	1975	2.30
DS4 (gnome)	SuperGlue	Huber	100 %	776	2.47
DS4 (gnome)	SuperGlue	Null	100 %	1392	2.44

3.3 Discussion of the numbers

A few observations follow directly from the tables.

++**Success rate.** The pipeline registers 100 % of the input frames in every cell of Tables 5 and 6. The success rate is therefore not a discriminator across our experiments, but it confirms that Task 3 (seed selection) and the next-best-view strategy do not abandon any frame even on the harder datasets. The convergence cost varies considerably: on **DS4** the seed loop tested up to 15 candidate pairs before accepting one, while on **DS2** the very first candidate was accepted.

Loss kernel comparison. The loss-kernel sweep (**Table 6**) reveals that the three kernels produce very similar results on the well-textured datasets (DS1, DS2, DS3) with ORB: error differences are within 0.02 px and density within a few percent. This indicates that on scenes with abundant, well-distributed features the outlier population is small enough that even standard least squares (Null) converges to a good solution. The picture changes on DS4 (gnome), where the sparse feature set makes the pipeline sensitive to the loss function: Cauchy produces the lowest error (0.55 px) but retains only 718 points, while Huber preserves the most points (2155) at a higher error (1.33 px), and Null collapses to 372 points. This confirms that on textureless subjects the robust kernel plays a significant role in balancing density against geometric precision.

For the SuperGlue pipeline, the kernel effect is more pronounced and dataset-dependent. On

DS1, all three kernels produce near-identical results (density within 13 points, error within 0.01 px), mirroring the ORB behaviour on this well-textured scene. On DS2, both Huber and Null reduce density compared to Cauchy (from 5945 down to 3263 and 3557 respectively), indicating that the less aggressive down-weighting causes bundle adjustment to converge to a geometry that the Task 7 outlier filter then prunes heavily. On DS3, Null produces the lowest SuperGlue error (1.29 px) but at the cost of halving density from 7563 to 3653, suggesting it discards many borderline matches that Cauchy manages to retain. On DS4, Cauchy is the clear winner: it preserves the most points (1975) at the lowest error (2.30 px), while Huber retains only 776 at 2.47 px and Null sits in between (1392 at 2.44 px).

The overall conclusion is that Cauchy is the best default kernel for both pipelines. On well-textured scenes the choice barely matters (and in some cells Null or Huber achieves a marginally lower error); on challenging scenes Cauchy offers the best density-to-error tradeoff, particularly for the noisier SuperGlue matches.

ORB versus SuperGlue. The baseline rows of **Table 5** show that ORB delivers a lower reprojection error than SuperGlue on every dataset. On DS2 (aloe) SuperGlue compensates with higher density (5945 versus 4687), and on DS4 (gnome) SuperGlue produces significantly more points (1739 versus 521). On the other datasets ORB wins on both metrics, most notably on DS3 (coffee cup) with 10468 versus 7563 points. Two factors contribute to SuperGlue’s higher error: first, the network accepts matches that ORB would reject as descriptor-Hamming outliers, because it reasons jointly about the assignment; second, the RANSAC threshold (0.001 in normalized units) was tuned for the corner-like keypoints ORB produces, and SuperPoint’s keypoint distribution differs.

The detailed impact of Optional 1 and Optional 2 on each dataset is discussed in Section 2.1.3 (per-dataset tables and analysis) and Section 2.1.4 (cross-dataset discussion of optional strategies).

3.4 Qualitative results and the impact of the input images

We present the qualitative evaluation organized by dataset. For each scene we show the point cloud produced by the ORB baseline, the SuperGlue baseline, and, where relevant, the configurations with Optional 1 and/or Optional 2 enabled. Green markers in all MeshLab screenshots indicate the estimated camera centers.

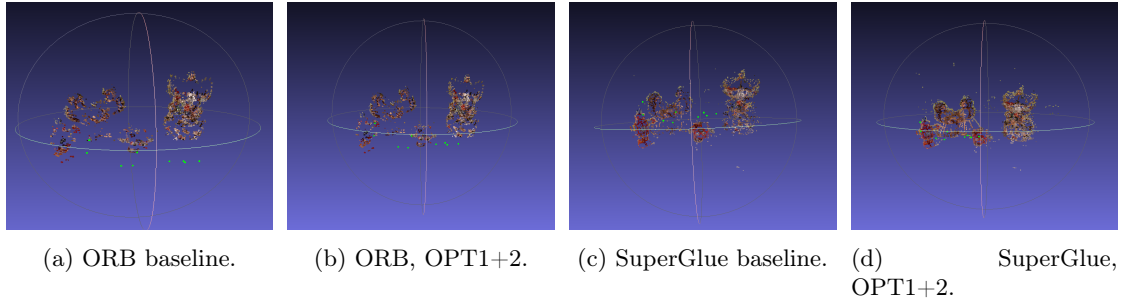


Figure 3: DS1 (angels): point cloud reconstructions under different configurations.

DS1 (angels). With the classical ORB pipeline, the reconstruction is decent: the four objects (angels statue, two robot toys, music box carousel) are identifiable, although they form disconnected sub-clusters because the white tabletop provides essentially no feature points to

bridge them. Enabling the optionals does not significantly alter this picture, since the geometric scoring of Optional 1 selects the same seed pair as the baseline on this dataset.

The SuperGlue pipeline produces a markedly better result. All four objects are distinguishable with noticeably denser point clouds per object, and the global structure is more coherent. The network’s ability to match subtle gradients on the smooth angel robes and the robot chassis, which ORB would reject as descriptor outliers, accounts for the density improvement. Activating both optionals on the SuperGlue matches preserves this quality: the reconstruction remains stable and visually comparable, confirming that the geometric seed scoring and pyramid-based next-best-view selection do not degrade an already well-conditioned initialization.

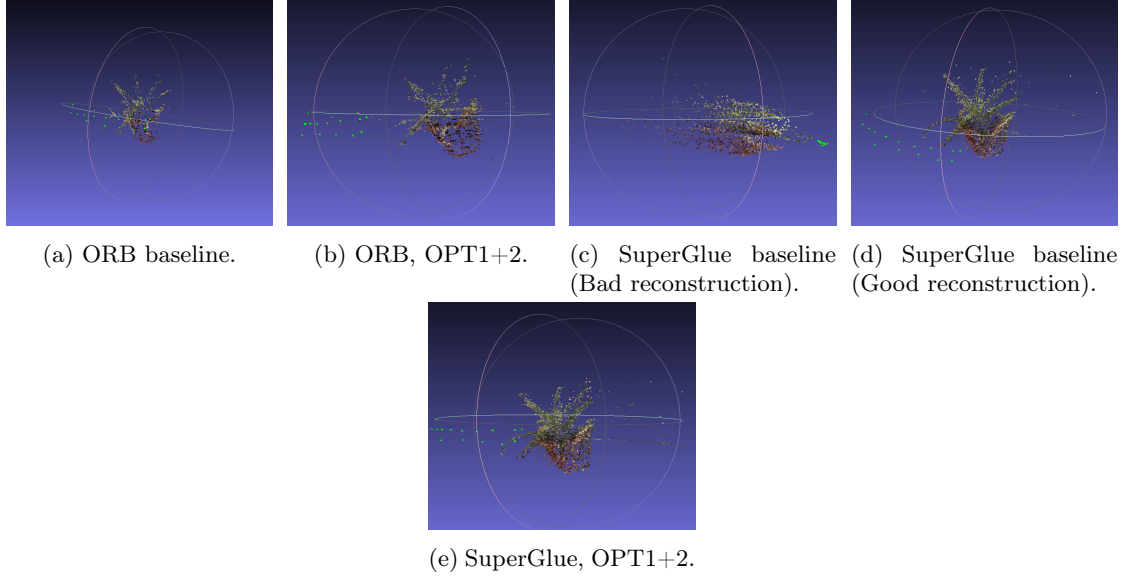


Figure 4: DS2 (aloe): point cloud reconstructions under different configurations.

DS2 (aloe). This is the easiest dataset for both pipelines. The leaves have natural high-frequency edges, the irregular geometry of the wicker basket adds further texture, and the green wall behind anchors a few background features. The ORB pipeline recovers a dense, stable reconstruction in which the plant silhouette and the pot are clearly recognizable, and enabling the optionals produces visually indistinguishable results.

The SuperGlue pipeline exhibits a bimodal behavior that is not reflected in the aggregate metrics. In approximately half of the executions, SuperGlue matches yield a reconstruction with even more vertices than ORB and a well-shaped plant.

In the other half, the reconstruction suffers from a characteristic distortion: the pot and lower portion of the plant are stretched along the optical axis, producing an elongated, unrealistic shape.

We attribute this instability to the way SuperGlue resolves ambiguous matches on the cylindrical pot surface, where the self-similar texture can lead to radially symmetric matching errors.

When these errors happen to align along the optical axis of the initial seed pair, the triangulated points are biased in depth, and bundle adjustment is unable to fully compensate because the error is systematic rather than random.

The ORB pipeline does not exhibit this failure mode because its Hamming distance threshold rejects the borderline matches that cause it.

Notably, enabling both optionals together eliminates the failure mode for the SuperGlue pipeline on this dataset. With Optional 1 active, the geometric seed scoring rejects the seed pairs that produce forward-dominant or planar-degenerate initial geometry, and with Optional 2 the pyramid-based view ordering ensures that subsequent cameras are registered from well-distributed viewpoints. The combination prevents the systematic depth bias from establishing itself in the first place, and the resulting reconstruction is consistently good across runs. This is one of the clearest cases in our evaluation where the optionals provide a qualitative, not merely quantitative, improvement.

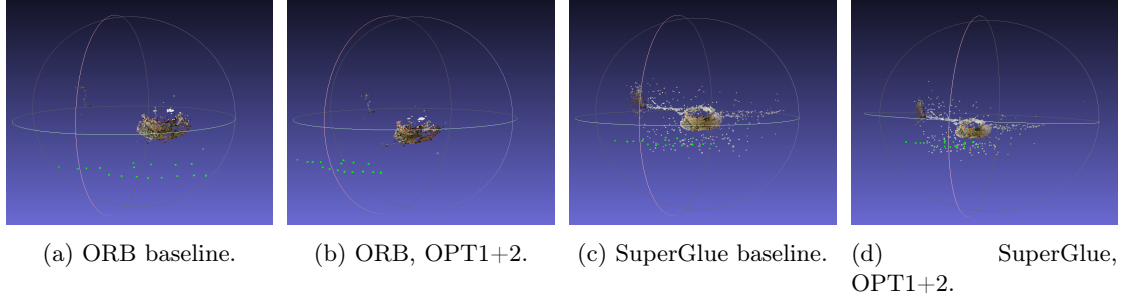


Figure 5: DS3 (coffee cup): point cloud reconstructions under different configurations.

DS3 (coffee cup). With the ORB pipeline, the reconstruction is generally good: the printed airmail design on the cup and saucer provides abundant, locally distinctive features, and the mug shape is recognizable. Enabling the optionals preserves this behavior, with the camera trajectory (green markers) confirming a well-distributed set of viewpoints around the object.

The SuperGlue pipeline is more variable. In favorable runs, it produces a result that is comparable to, or slightly better than, the ORB baseline, with a denser coverage of the cup surface. In less favorable runs, the cloud exhibits a noticeable halo of spurious vertices scattered around the cup. In the worst cases, this noise can exceed that of the ORB pipeline, although the overall mug shape remains recognizable even in the noisier executions. The cause of the noise is twofold: the saucer’s circumferential symmetry allows the network to occasionally match a border feature to its symmetric counterpart at a different angular position, and the repetitive postage stamp motifs on the cup produce locally ambiguous descriptors.

These spurious correspondences survive SuperGlue’s internal outlier rejection because their epipolar residual is small (the symmetric counterpart lies near the true epipolar line), but they introduce depth errors that manifest as the observed halo. Enabling both optionals does not eliminate this phenomenon, since it originates in the matching stage rather than in the seed selection or view ordering.

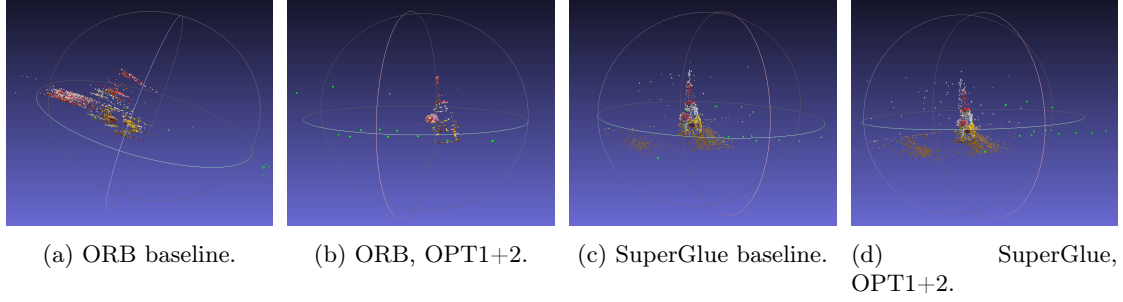


Figure 6: DS4 (gnome): point cloud reconstructions under different configurations.

DS4 (gnome). This is the most challenging dataset due to the large textureless regions on the gnome body (uniform red, white, and yellow paint) and the featureless background. With the ORB baseline, the frontal face of the gnome is decently reconstructed, since the face, belt, and small mushroom provide enough corners for ORB.

However, the profile view reveals a severe distortion along the optical axis: the few feature points on the smooth sides of the body are triangulated with poor depth precision, stretching the cloud into an elongated shape. This is a direct consequence of the narrow effective baseline that ORB features provide on surfaces with weak texture.

Enabling one or both optionals produces a clear improvement in the ORB case. The geometric seed scoring of Optional 1, combined with the pyramid-based view ordering of Optional 2, selects an initialization and a registration sequence that yield a more uniformly sampled cloud. The gnome shape is correctly reproduced, even if the level of detail remains limited by the sparse feature count.

The SuperGlue pipeline on DS4 exhibits the most extreme bimodality we observed in the entire evaluation. In approximately half of the executions, the reconstruction is excellent: the gnome is rendered with good coverage and recognizable proportions, exploiting the subtle gradient features that SuperPoint extracts from the smooth painted surfaces.

In the other half, the reconstruction fails catastrophically, producing either a severely distorted or collapsed point cloud.

The explanation for this bimodality lies in the sensitivity of the incremental pipeline to the initial two-view geometry on a subject with very few reliable correspondences. When the seed pair happens to produce a well-conditioned triangulation, the entire chain of subsequent registrations and bundle adjustments benefits from it, and the final cloud is high quality.

When the seed pair is poorly conditioned (even marginally), the initial depth errors propagate through every subsequent registration, and bundle adjustment cannot recover because the robust loss has too few correctly triangulated points to anchor the solution.

3.5 Summary of qualitative observations

The qualitative results confirm that reconstruction quality depends strongly on the interplay between scene texture and matching strategy. The aloe (DS2) is reliably easy for all configurations. The angels (DS1) benefit most visibly from the SuperGlue pipeline, which produces the most complete reconstruction of all four objects. The coffee cup (DS3) exposes SuperGlue’s vulnerability to circumferential symmetry, manifesting as a noisy halo in unfavorable runs. The gnome (DS4) shows the largest qualitative gap between configurations: the ORB baseline with optionals enabled is more stable than the ORB baseline alone, while SuperGlue can produce the best or the worst result depending on the stochastic initialization.

For the lab’s quantitative metric (mean reprojection error), ORB with the Cauchy kernel offers the best density-to-error tradeoff on every dataset; for visual completeness on textureless scenes, SuperGlue in its favorable mode is the better choice.

4 Group members and contributions

The work was distributed as follows: (**Francesco Carretta**) implemented Task 1 (geometric verification) and Task 2 (the SuperGlue Python script and the three C++ helper functions for loading external matches), and ran the camera calibration procedure for the smartphone datasets. (**Giuseppe D’Auria**) implemented Task 3 (seed pair selection) and Task 4 (triangulation), implemented Task 5 (residual functor), Task 6 (bundle adjustment with the configurable loss kernel), Task 7 (divergence handling and the final metrics block) and performed the evaluation with different Kernels.

(**Anna Mainardis**), implemented : OPTIONAL 1 and OPTIONAL 2.

The writing of this report was a joint effort by the three members.

References

- [1] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “ORB: An efficient alternative to SIFT or SURF,” in *IEEE ICCV*, 2011.
- [2] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-supervised interest point detection and description,” in *CVPR Workshops*, 2018.
- [3] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperGlue: Learning feature matching with graph neural networks,” in *IEEE CVPR*, 2020.
- [4] S. Agarwal, K. Mierle, et al., “Ceres Solver,” <http://ceres-solver.org>.
- [5] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge Univ. Press, 2004.